Experiment – 1 b: TypeScript

Name of Student	Anushka Shahane
Class Roll No	**54**
D.O.P.	30/1/25
D.O.S.	6/2/25
Sign and Grade	

Aim: To study Basic constructs in TypeScript.

Problem Statement:

a. Create a base class **Student** with properties like name, studentId, grade, and a method getDetails() to display student information.
Create a subclass **GraduateStudent** that extends Student with additional properties like thesisTopic and a method getThesisTopic().
   - Override the getDetails() method in GraduateStudent to display specific information.

Create a non-subclass **LibraryAccount** (which does not inherit from Student) with properties like accountId, booksIssued, and a method getLibraryInfo().
Demonstrate composition over inheritance by associating a LibraryAccount object with a Student object instead of inheriting from Student.
Create instances of Student, GraduateStudent, and LibraryAccount, call their methods, and observe the behavior of inheritance versus independent class structures.

1. **Output :**

```
class Student {
name: string;
studentId: string;
grade: string;
constructor(name: string, studentId: string, grade: string) {
this.name = name;
this.studentId = studentId;
this.grade = grade;
}
getDetails(): string {
return `Name: ${this.name}, Student ID: ${this.studentId}, Grade: ${this.grade}`;
}
}
class GraduateStudent extends Student {
thesisTopic: string;
constructor(name: string, studentId: string, grade: string, thesisTopic: string) {
super(name, studentId, grade);
this.thesisTopic = thesisTopic;
}
getDetails(): string {
return `${super.getDetails()}, Thesis Topic: ${this.thesisTopic}`;
}
```

```
getThesisTopic(): string {
return `Thesis Topic: ${this.thesisTopic}`;
}
}
class LibraryAccount {
accountId: string;
booksIssued: number;
constructor(accountId: string, booksIssued: number) {
this.accountId = accountId;
this.booksIssued = booksIssued;
}
getLibraryInfo(): string {
return `Account ID: ${this.accountId}, Books Issued: ${this.booksIssued}`;
}
}
class StudentWithLibraryAccount {
student: Student;
libraryAccount: LibraryAccount;
constructor(student: Student, libraryAccount: LibraryAccount) {
this.student = student;
this.libraryAccount = libraryAccount;
}
getFullDetails(): string {
```

```
return `${this.student.getDetails()}, ${this.libraryAccount.getLibraryInfo()}`;

}

}

const student1 = new Student("Rina", "S12345", "A");

console.log(student1.getDetails());

const gradStudent1 = new GraduateStudent("Anushka Shahane", "G67890", "A+", "UX Design");

console.log(gradStudent1.getDetails());

const libraryAccount1 = new LibraryAccount("L98765", 5);

console.log(libraryAccount1.getLibraryInfo());

const studentWithLibraryAccount = new StudentWithLibraryAccount(student1, libraryAccount1);

console.log(studentWithLibraryAccount.getFullDetails());

const gradStudentWithLibraryAccount = new StudentWithLibraryAccount(gradStudent1, libraryAccount1);

console.log(gradStudentWithLibraryAccount.getFullDetails());
```

```
C:\Users\Student.DESKTOP-HH21O6Q\Desktop\folder>node library.js
Name: Rina, Student ID: S12345, Grade: A
Name: Anushka Shahane, Student ID: G67890, Grade: A+, Thesis Topic: UX Design
Account ID: L98765, Books Issued: 5
Name: Rina, Student ID: S12345, Grade: A, Account ID: L98765, Books Issued: 5
Name: Anushka Shahane, Student ID: G67890, Grade: A+, Thesis Topic: UX Design, Account ID: L98765, Books Issued: 5

C:\Users\Student.DESKTOP-HH21O6Q\Desktop\folder>
```

B]

Design an employee management system using TypeScript. Create an Employee interface with properties for name, id, and role, and a method getDetails() that returns employee details. Then, create two classes, Manager and Developer, that implement the Employee interface. The Manager class should include a department property and override the getDetails() method to include the department. The Developer class should include a programmingLanguages array property and override the getDetails() method to include the programming languages. Finally, demonstrate the solution by creating instances of both Manager and Developer classes and displaying their details using the getDetails() method.

```
interface Employee {

name: string;

id: string;

role: string;

getDetails(): string;

}

class Manager implements Employee {

name: string;

id: string;

role: string;

department: string;

constructor(name: string, id: string, department: string) {

this.name = name;

this.id = id;

this.role = 'Manager';

this.department = department;
```

```
}

getDetails(): string {

return `${this.name} (ID: ${this.id}) is a ${this.role} in the ${this.department} department.`;

}

}

class Developer implements Employee {

name: string;

id: string;

role: string;

programmingLanguages: string[];

constructor(name: string, id: string, programmingLanguages: string[]) {

this.name = name;

this.id = id;

this.role = 'Developer';

this.programmingLanguages = programmingLanguages;

}

getDetails(): string {

return `${this.name} (ID: ${this.id}) is a ${this.role} skilled in ${this.programmingLanguages.join(', ')}.`;

}

}

const manager1 = new Manager('Anushka Shahane', 'M001', 'Sales');

const developer1 = new Developer('Rina', 'D001', ['JavaScript', 'TypeScript', 'Python']);
```

```
console.log(manager1.getDetails());

console.log(developer1.getDetails());
```

```
C:\Users\Student.DESKTOP-HH21O6Q\Desktop\folder>tsc employee.ts

C:\Users\Student.DESKTOP-HH21O6Q\Desktop\folder>node employee.js
Anushka Shahane (ID: M001) is a Manager in the Sales department.
Rina (ID: D001) is a Developer skilled in JavaScript, TypeScript, Python.

C:\Users\Student.DESKTOP-HH21O6Q\Desktop\folder>
```

Github Link : https://github.com/Anushka3204/WebX_Experiment_1B

Conclusion : This experiment helped in understanding the core object-oriented concepts of TypeScript, such as inheritance, interfaces, and composition. By building student and employee management systems, we explored how to structure scalable and modular code using classes and interfaces.